RUFO - Rescued and Unidentified Fluffy Object

Murillo Cortes Jeanette, y equipo de proyecto

CENTRO UNIVERSITARIO DE CIENCIAS
EXACTAS E INGENIERÍAS, (CUCEI, UDG)

***Abstract*—La iniciativa aborda la sobrepoblación de perros sin hogar en Guadalajara, México, a través de la creación del sitio web RUFO. Se destaca la alarmante cantidad de perros callejeros en México y por qué esta razón se refleja en la seguridad ciudadana. El desarrollo del proyecto integra tecnologías como Javascript, React, Node.Js, Express, PostgreSQL, TypeORM y Docker para el frontend y backend del sitio web. Resultados clave incluyen un catálogo de mascotas funcional, una gestión eficiente de las mascotas en albergues y conciencia pública sobre los perros callejeros. Las conclusiones resaltan el impacto positivo en la adopción de perros, el bienestar animal y la adopción responsable. Este proyecto ofrece una experiencia valiosa para proyectos futuros que busquen abordar problemas sociales críticos con un enfoque centrado en la seguridad.**

***Palabras claves* – Adopción responsable, sistema recomendador, sitio web, Javascript, React, Node.Js, Express, PostgreSQL, TypeORM, Docker, impacto social.**

Versión actual del código: Versión 1.0

I. Introducción

El proyecto representa una respuesta a una problemática crucial que afecta a la sociedad actual: la sobrepoblación de perros sin hogar en Guadalajara, Jalisco. A medida que la población de perros callejeros sigue creciendo, surge la necesidad urgente de abordar este problema y encontrar soluciones efectivas para mejorar la calidad de vida de estos animales, promover la adopción responsable y disminuir la probabilidad de agresiones que sufren ellos en la calle.

La estrategía para este proyecto, ha sido dirigirse a los lugares u organizaciones que apoyan a la sociedad a afrontar la problemática, los albergues, organizaciones dedicadas a brindar refugio y cuidado a perros sin hogar. Los cuales, a pesar de sus esfuerzos incansables, se enfrentan a la difícil tarea de rescatar animales de las calles y encontrar hogares amorosos para garantizarles su bienestar.

El proyecto tiene como población beneficiaria tanto a los perros sin hogar que residen en los albergues como a la seguridad de la comunidad de Guadalajara y sus alrededores. El bienestar de los perros, la gestión de la población canina y la educación sobre la adopción responsable son aspectos cruciales que abordaremos.

La solución propuesta es la creación de un sitio web integral llamado "RUFO" (Rescued and Unidentified Fluffy Object) que servirá como plataforma de adopción en línea y una herramienta de gestión interna para albergues. Los objetivos principales abarcan el aumento de las adopciones mensuales, la mejora de la gestión interna de albergues en Guadalajara, la creación de conciencia sobre la importancia de la adopción responsable y la facilitación de una mejor seguridad ciudadana, asegurando un entorno seguro para las mascotas rescatadas y para aquellos que las adoptan mediante esta herramienta de gestión para albergues.

II. Trabajos Relacionados

En la búsqueda de soluciones a la sobrepoblación de perros sin hogar y la promoción de la adopción responsable, se han identificado varios proyectos en curso, entre los cuales podemos resaltar:

A. *PetFinder*

Es una de las aplicaciones móviles líderes para la conexión de adoptantes y mascotas necesitadas. Su plataforma permite buscar, filtrar y adoptar animales de refugios y rescates. Su interfaz ofrece perfiles detallados y fotos, promoviendo la adopción ética, la diversidad de opciones y la posibilidad de salvar vidas. Sin embargo, algunos contras pueden surgir de la variabilidad en la calidad de la información y desafíos logísticos en el proceso de adopción.

B. *AdoptaFácil*

AdoptaFácil es un sitio web que conecta a refugios de animales con posibles adoptantes en línea. Su interfaz permite

buscar y filtrar animales según preferencias y ubicación, es por ello que si bien este proyecto aborda parcialmente la problemática, aún hay espacio para mejoras en términos de eficiencia al coordinar las adopciones, pues se mantienen algo limitadas por la disponibilidad de los refugios registrados.

C. *GuadaPets*

Es una aplicación móvil que permite a los usuarios buscar perros para adoptar en Guadalajara, diseñada para simplificar el cuidado de las mascotas. Esta aplicación permite una administración eficaz de la salud, permitiendo la conectividad con servicios veterinarios. Sin embargo, su alcance llega a ser limitado, dado que no aborda la gestión interna del albergue.

D. *Adop-a-Pet.com*

Adopt-a-Pet.com (2000) es una de las plataformas digitales líder en la adopción de mascotas. Facilita la conexión entre animales en busca de hogar y adoptantes potenciales. Tiene una extensa base de datos de perfiles de mascotas, con detalles exhaustivos y fotografías. A pesar de ser bastante útil y ofrecer buenas opciones, al no tener implementado un sistema de recomendaciones personalizadas basado en preferencias del adoptante, se vuelve complicado para un usuario promedio elegir una mascota para adoptar.

III. Descripción del Desarrollo del Proyecto Modular

A. Metodología de Trabajo

El equipo adoptó una metodología ágil, basada en los principios de Scrum. Se organizaron sprints de semanas, donde se planificaron y ejecutaron las tareas necesarias para alcanzar los objetivos específicos. Se llevaron a cabo reuniones semanales de un día, a veces dos días a la semana de seguimiento para mantener la comunicación y la coordinación del proyecto en el equipo.

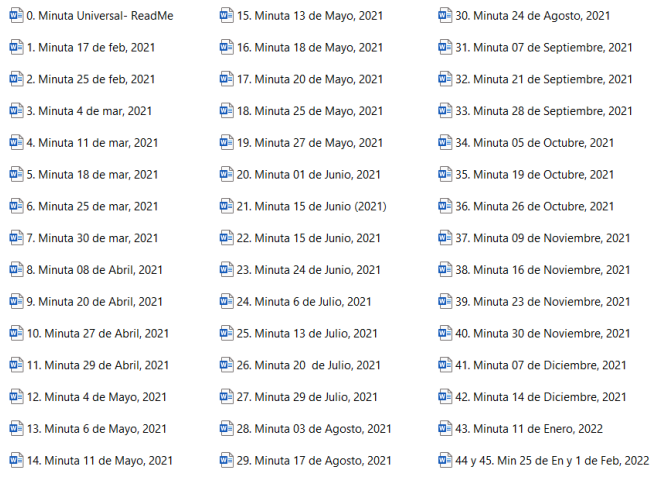


Fig. 1. Documentos de las primeras minutas de los sprints

La metodología para la realización del proyecto se dividió en las siguientes fases:

- Planificación: En esta etapa, se obtuvieron los requisitos, de manera que estuvieran construidos sobre las necesidades de los posibles adoptantes y personal de los albergues.
- Desarrollo: Se programaron las funcionalidades clave, en línea con la metodología ágil y entregas incrementales, en base a los requisitos previamente obtenidos, asegurando la alineación constante con las necesidades del proyecto
- Pruebas y Depuración: Se realizaron pruebas funcionales utilizando casos de prueba con los cuales se buscaba obtener el resultado esperado, esto con el fin de identificar posibles errores durante el proceso.
- Mantenimiento: Se estableció un plan para abordar actualizaciones y mejoras futuras.

El desarrollo de "RUFO" se basó en la colaboración constante entre el equipo de desarrollo, refugios de animales y posibles adoptantes, con el objetivo de crear una plataforma efectiva y amigable para promover la adopción responsable de perros sin hogar.

B. Requerimientos Principales

Los requerimientos principales de "RUFO" se definieron a partir de la identificación de las necesidades clave. A continuación se resumen los puntos más importantes:

1. Plataforma en Línea: Crear una plataforma web accesible desde cualquier dispositivo con conexión a Internet.
2. Presentación de la página: Datos que se presentarán a primera vista en el Sitio Web: Adopta, Dona, Acerca de nosotros, Voluntariado para generar un interés y una consciencia respecto a la adopción responsable.
3. CRUD de Mascotas: Módulo para capturar la información general de un perro al ingresar al albergue para el control poblacional en una base de datos para su consulta y procesamiento posterior para editar y eliminar una mascota.
4. Búsqueda Avanzada: Permitir a los usuarios buscar perros disponibles para adopción según criterios específicos, como raza, tamaño y personalidad.
5. Registro de Personal: Facilitar la capacidad de registrar y gestionar perfiles de los perros disponibles para adopción, además de los usuarios del personal que administre la coordinación de las adopciones.
6. Perfil de Usuario: Ofrecer a los usuarios la posibilidad de crear perfiles personales, guardar perfiles de perros favoritos y enviar múltiples solicitudes de adopción.

7. Solicitud de adopción: Cuestionario de solicitud de adopción en línea que se pueda consultar desde los perfiles del personal administrador.

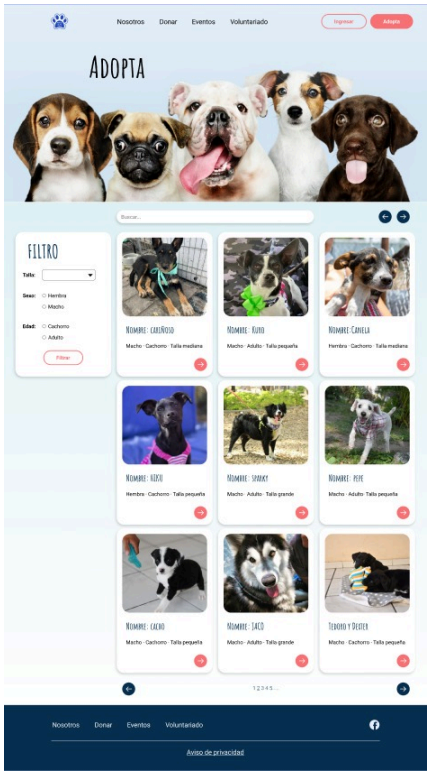


Fig. 2. Vista del Sitio Web en Adopta.

C. Impacto Psicológico de los Colores

El sitio web buscaba que los adoptantes percibieran de manera positiva y cálida la página web. Para transmitir a los usuarios sensaciones de empatía y amor, se utilizaron los siguientes colores:

- Color rosa bajito: Utilizado para transmitir calidez, cuidado y afecto, creando una conexión emocional con los usuarios.
- Color blanco: Empleado para equilibrar la presencia del rosa bajito, asegurando que el diseño se mantenga limpio y sencillo, además de transmitir pureza y simplicidad.
- Color azul marino: Seleccionado para evocar sentimientos de confianza, seguridad y profesionalismo, promoviendo una sensación de estabilidad y confiabilidad en el usuario.

Estos colores en conjunto crean un ambiente visual que busca generar empatía, amor y conexión, haciendo que los usuarios se sientan bienvenidos y valorados al interactuar con nuestra plataforma. Además, crean una primera impresión favorable que capta la atención de los usuarios y fomenta una conexión emocional.

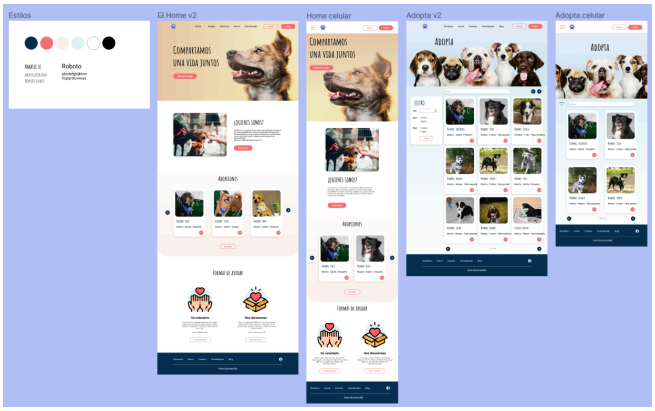


Fig. 3. Paleta de colores y vista de la segunda versión del diseño del Sitio Web creada en Figma.

El enfoque en la selección de colores se basó en el análisis de mapas de empatía y en varias versiones del diseño, garantizando que las percepciones y necesidades de los usuarios se reflejaran adecuadamente en el sitio web.

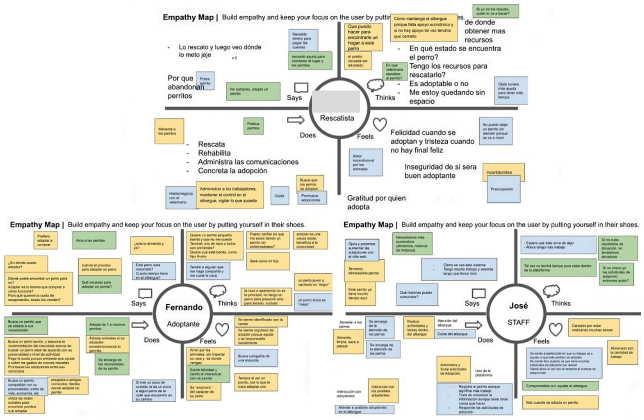


Fig. 4. Emphaty Maps para Rescatistas, Adoptantes y Staff.

D. Tecnologías Utilizadas

Para el desarrollo de "RUFO", se utilizaron las siguientes tecnologías:

- Control de versiones: GitHub como repositorio público del código del proyecto, y TypeORM para el control de versiones de la base de datos.
- Base de Datos: Empleando PostgreSQL para el almacenamiento de datos.
- Backend: Implementando Node.js con Express para la lógica del servidor.
- Frontend: Utilizando React.js con HTML, JS y CSS para la creación de la interfaz de usuario.

- Contenedores: Docker para la contenerización del Front-End, Back-End y Base de Datos.

E. Pruebas Realizadas

Se realizaron pruebas periódicas para garantizar el correcto funcionamiento del sistema, mediante pruebas de rendimiento y seguridad. Se informaba del estado de los requerimientos cada vez que se detectaba un error. Cada error se clasificaba como tiquets de 'Baja', 'Alta' o 'Media' prioridad, y se abordaban en los sprints. Además, se llevaron a cabo pruebas de aceptación con usuarios reales para obtener retroalimentación valiosa y realizar mejoras iterativas. Estas prácticas aseguraron un desarrollo sólido y una experiencia de usuario óptima, reforzando la calidad y fiabilidad del producto final.

F. Proceso de Implementación

El proceso de implementación de "RUFO" se dividió en varias fases:

1. Diseño: Se crearon diferentes diseños de la interfaz de usuario en la plataforma de Figma y se definieron los diagramas que permitieran describir el sistema en su entorno de manera lógica.
2. Desarrollo: Se implementaron las funcionalidades clave siguiendo la metodología ágil, con entregas incrementales que permitieron tener entregas de avances continuos.
3. Pruebas y Depuración: Se realizaron pruebas funcionales y se corrigieron los errores identificados, controlando los cambios e informando periódicamente sobre el estado de los requerimientos.
4. Aceptación: Aprobado el software en base a su cumplimento con los requerimientos, se desplegó la plataforma en un entorno de producción accesible al público.
5. Mantenimiento Continuo: Se estableció un plan de mantenimiento de manera periódica para abordar actualizaciones y mejoras futuras.

El desarrollo de "RUFO" se basó en la colaboración constante entre el equipo de desarrollo y los posibles adoptantes, con el objetivo de crear una plataforma efectiva y amigable para promover la adopción responsable de perros sin hogar.

Módulo I Justificación de Arquitectura y Programación de Sistemas

El primer módulo se centra en la justificación de la arquitectura y la programación de sistemas utilizados. A continuación, se detallan las acciones llevadas a cabo:

1.1 Lenguajes de Programación

El desarrollo del proyecto en el backend se llevó a cabo con Node.js y JavaScript, seleccionados por su eficiencia en la gestión de solicitudes HTTP, su facilidad de despliegue y naturaleza liviana facilitan la escalabilidad en entornos de producción. En cuanto al frontend, se optó por React.js debido a su capacidad para una arquitectura encapsulada, lo que posibilita un mantenimiento de código claro y sencillo.

1.2 Bases de Datos y/o Estructuras de Datos

El proyecto se desarrolló en PostgreSQL para el almacenamiento de datos, debido a su robustez y eficiencia en el manejo de datos estructurados, destacando por su capacidad para abordar consultas complejas. La elección de PostgreSQL se basó en su capacidad para almacenar de manera efectiva los perfiles de perros y usuarios, estableciendo así una base sólida y confiable para la gestión de datos en el proyecto.

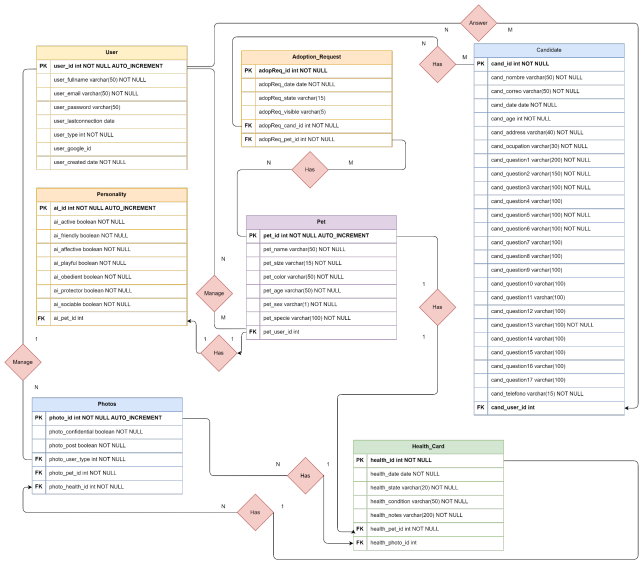


Fig. 5. Diagrama Entidad Relación general.

1.3 Metodología de Programación a Seguir

Se adoptó una metodología ágil basada en Scrum para el desarrollo. Esta elección permitió una planificación y ejecución efectiva de tareas, iteraciones constantes y adaptación a las necesidades cambiantes del proyecto. Scrum se destacó por su enfoque colaborativo y transparente, fomentando la comunicación continua entre el equipo. La estructura de responsabilidades definidas facilitó una distribución equitativa de tareas y una mayor eficiencia en el trabajo en equipo. Además, la capacidad de Scrum para identificar y abordar rápidamente los obstáculos y riesgos del proyecto permitió garantizar una gestión ágil de los cambios y una rápida resolución de problemas.

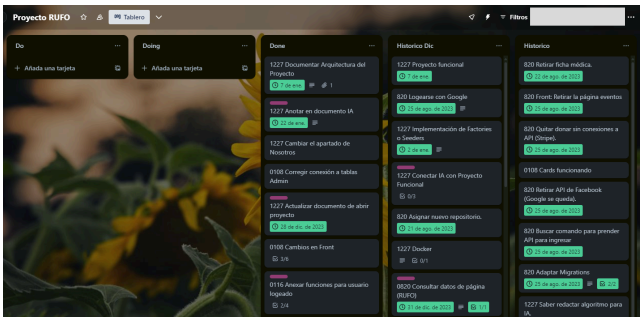


Fig. 6. Tablero de SCRUM del proyecto.

1.4 Elementos de Ingeniería de Software

En el ámbito de la ingeniería de software, se ha dado máxima prioridad a la eficiencia, escalabilidad y usabilidad en cada fase del proceso. Desde la definición de los requisitos hasta el diseño de la arquitectura, la codificación, las pruebas, la documentación y el despliegue, cada elemento ha sido abordado con el objetivo de asegurar un desarrollo eficaz que permita al proyecto un buen rendimiento, capacidad de crecimiento y facilidad de uso.

En cuanto a la arquitectura del proyecto, este cuenta con un ORM que gestiona las versiones de la base de datos. La base de datos PostgreSQL, a su vez, envía información a la API que incluye el CRUD de la página web, el algoritmo de inteligencia artificial y la autenticación con Google. Esto permite la utilización y visualización de estos datos desde el frontend de la página web.

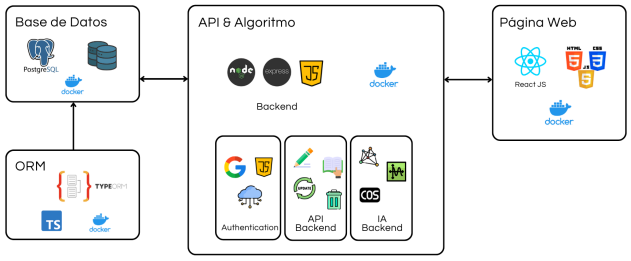


Fig. 7. Diagrama de la arquitectura del Proyecto.

1.5 Modelado del Sistema

Se crearon Diagramas de Casos de Uso, Diagramas de Entidad-Relación, Diagramas de Flujo, Mapas de Empatía para la construcción del sistema "RUFO". Esta metodología permitió una mejor comprensión y visualización de la arquitectura del proyecto, lo que agilizó tanto su desarrollo como su mantenimiento. A continuación, se presentan los diagramas utilizados en la modelización del sistema.

El diagrama de flujo describe el proceso de seguimiento al ingresar en el sitio web, comenzando con la selección de opciones en la página principal, que incluyen "Adopta", "Acerca de Nosotros", "Donar" y "Voluntariado". Si se elige el apartado "Adopta", se muestran todas las mascotas disponibles para adopción. En este apartado, el usuario puede explorar las mascotas o pedir recomendaciones, que mostrarán mascotas sugeridas. Una vez seleccionada una mascota de interés, se puede ver su información y elegir entre "Añadir a favoritos" o "Enviar una solicitud de adopción". Para estas opciones, es necesario que el usuario cree una cuenta o inicie sesión con una cuenta existente. Después de agregar un favorito o solicitar una mascota, el usuario puede regresar a explorar más mascotas.

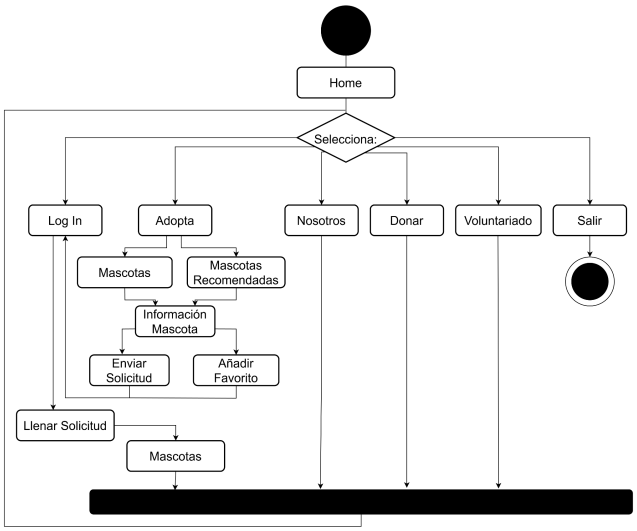


Fig. 8. Diagrama de Flujo.

En cuanto a los diagramas de Caso de Uso, el primer tipo de actor considerado para el sistema es el Usuario Visitante. Este tipo de actor, puede acceder al sitio web, para de esta manera ingresar a los distintos apartados desarrollados para RUFO, como lo son el Home page inicialmente, además de Adopta, Nosotros, Donar, Eventos y Voluntariado. Este usuario visitante puede ver los adoptables de perritos, pedir recomendaciones, y ver los demás apartados del sitio Web, sin embargo, para solicitar una solicitud de adopción tiene que crear una cuenta.

El segundo tipo de actor considerado es el Usuario Adoptante, el cual puede realizar las interacciones del Usuario Visitante, con la particularidad de que este segundo actor puede ingresar a la página con una cuenta que le permita ver las mascotas que ha solicitado en adopción como así también sus mascotas Favoritas. Es por esto que cuando ingresa en el sistema, se va a realizar una validación en la Base de Datos para verificar que es un usuario existente.

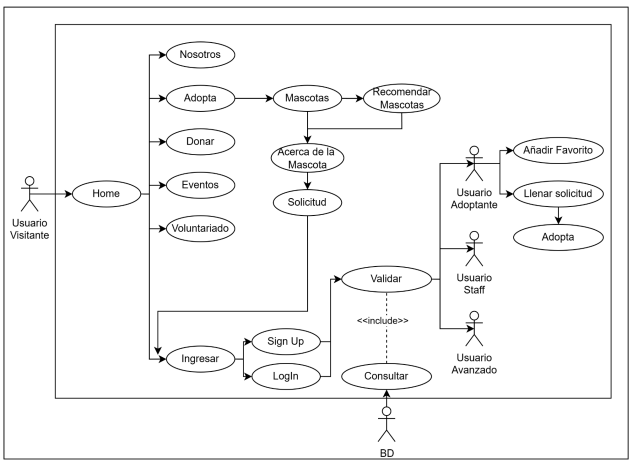


Fig. 9. Caso de Uso para usuarios Visitante y Adoptante.

El tercer y cuarto tipo de actores, son el Usuario Staff y el Usuario Avanzado, correspondientemente. Estos, pueden realizar las mismas acciones que el Usuario Visitante, y además ingresar con una cuenta al sistema. Dicha cuenta, se va a verificar sea existente en la Base de Datos, para de esta manera, se permita acceder a las funcionalidades adicionales que pueden estos actores realizar en el sistema.

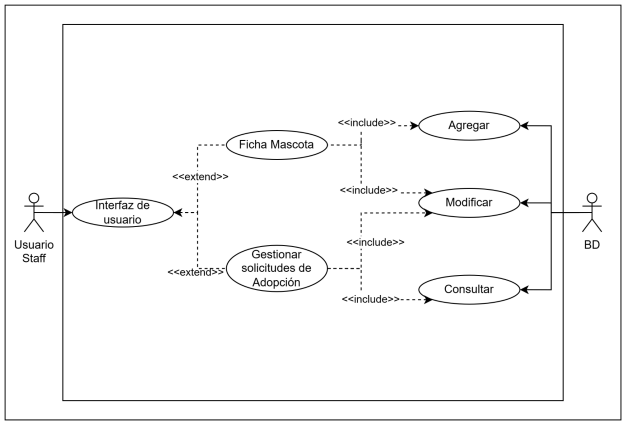


Fig. 10. Caso de Uso para usuario Staff.

En el segundo Diagrama de Caso de Uso, muestra el tercer tipo de actor, el Usuario Staff, el cual es un trabajador del albergue Patitas de la Calle. Este actor puede realizar acciones de administración y consulta para el refugio, como lo son las funciones de Agregar, Modificar y Consultar la ficha de una mascota. También puede realizar consulta de solicitudes de adopción de una mascota, modificando el estado en el que se encuentra dicha solicitud.

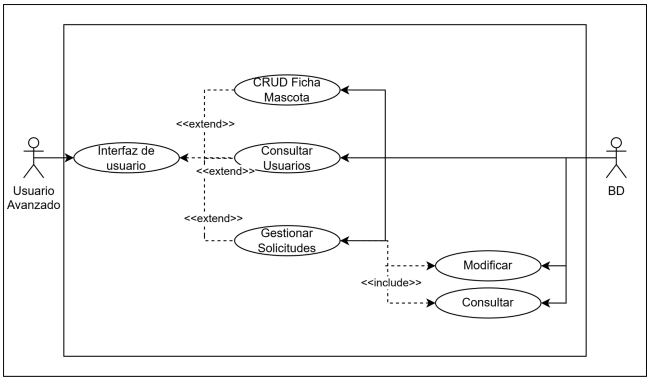


Fig. 11. Caso de Uso para usuario Avanzado.

Por su parte, el cuarto actor, el Usuario Avanzado, se considera un administrador y puede realizar un CRUD (Create, Read, Update, Delete) para una Ficha de Mascota, pudiendo visualizar de igual manera a los Usuarios registrados en el sistema. Además, también puede gestionar las solicitudes de adopción para modificarlas y consultarlas, con la posibilidad de 'Rechazar' o 'Finalizar' las solicitudes que se han realizado por medio del Sitio Web.

Modulo II Justificación de Sistemas Inteligentes

En esta etapa, se implementaron sistemas inteligentes, destacando el uso del Aprendizaje Automático (Machine Learning) para sugerir perros basándose en el sexo, talla, edad y personalidades (activo, amigable, amoroso, juguetón, obediente, protector, sociable, tierno, tímido, tranquilo), introduciendo la técnica de similitud de cosenos para mejorar la eficacia de las recomendaciones.

Se seleccionó este algoritmo debido a su capacidad para entrenarse inicialmente y, con el tiempo, generar recomendaciones basadas en su aprendizaje continuo, eliminando la necesidad de reentrenamiento frecuente.

Es importante mencionar que la aplicación de la inteligencia artificial al comienzo del proyecto utilizaba el algoritmo k-Vecinos más cercanos (k-NN), sin embargo se ampliaron las recomendaciones utilizando la técnica de suma de cosenos, dado que esta aproximación permite evaluar de manera más precisa la similitud entre las preferencias de los usuarios y las características de los perros, mejorando la calidad de las sugerencias.

La decisión de cambiar del algoritmo k-vecinos más cercanos (k-NN) a la suma de cosenos se basó en la observación de su rendimiento superior, especialmente a medida que los datos aumentaban. Esta adaptación proporciona recomendaciones altamente personalizadas y más afinadas.

La elección de la suma de cosenos se fundamenta en su capacidad para evaluar la orientación y la similitud direccional

entre vectores de preferencias, brindando una perspectiva más sofisticada en la generación de recomendaciones personalizadas.

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}},$$

Fig. 12. Fórmula para calcular la similitud.

Para el proyecto, el sistema funciona de la siguiente manera: en la sección "Adoptar" del sitio web, en la parte inferior derecha, hay un botón denominado "Recomendar", que permite a los usuarios seleccionar sus preferencias al buscar una mascota.

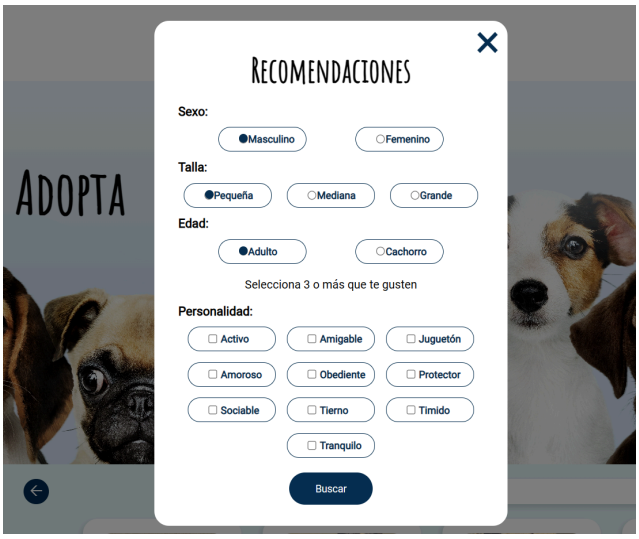


Fig. 13. Selección de características a recomendar.

Una vez que el usuario selecciona sus preferencias, esta información se convierte en un vector. Al hacer clic en el botón "Buscar", la similitud de coseno compara este vector con los vectores almacenados en la base de datos. Los resultados se ordenan según su proximidad al valor "1", indicando que los valores más cercanos representan mejores recomendaciones, las cuales se muestran en la página.

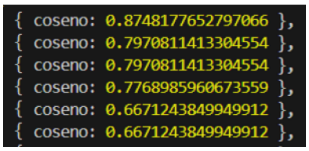


Fig. 14. Primeros resultados de una recomendación.

A medida que aumentaban los datos, la similitud de coseno ofrecía recomendaciones superiores en comparación con otros algoritmos, gracias a su capacidad para gestionar grandes volúmenes de datos y proporcionar resultados más precisos.

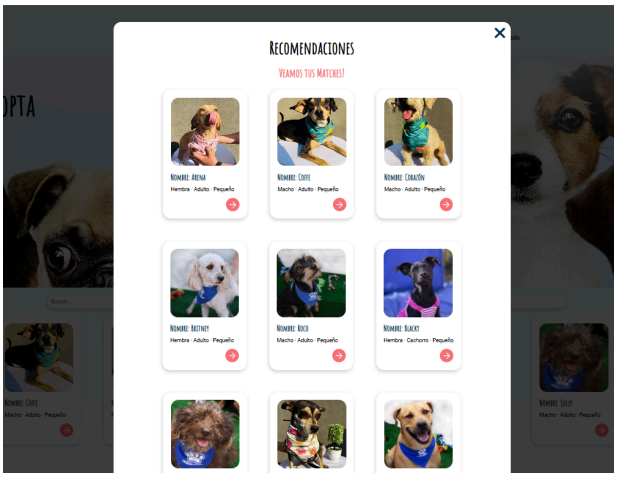


Fig. 15. Matches de perritos mostrados al usuario.

Con base en estos resultados, se habilitó la visualización de los posibles "Matches" de perros para el usuario, considerando la personalidad y las características que hayan seleccionado previamente. De entre estos candidatos, se identificaron los 25 mejores compatibles que podrían ajustarse a las preferencias del usuario, proporcionando así opciones que se alinean con sus criterios de búsqueda.

Módulo III Justificación de Sistemas Distribuidos

En la implementación del proyecto, se integró Docker como herramienta de contenedores para alcanzar un sistema que garantice la continuidad del servicio en situaciones adversas, lo que facilita la migración y la recuperación en caso de fallos.

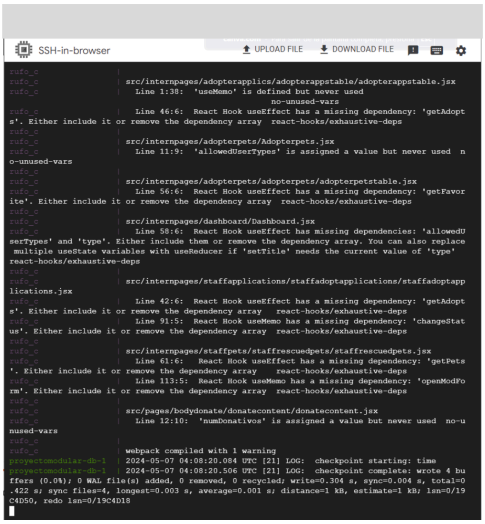


Fig. 16. Imagen de Docker.

La captura muestra una terminal SSH donde se está ejecutando el proyecto "docker-rufo-vm". En esta instancia, se está llevando a cabo una compilación o verificación del código del proyecto "proyectomodular-db-1".

En la configuración de la máquina virtual, se utilizaron diversas imágenes de Docker para asegurar una integración fluida. Estas comprenden cuatro imágenes de Docker para los siguientes componentes:

- Servidor web (rufo)
- API, incluye algoritmo (api)
- Base de datos PostgreSQL (db)
- ORM (orm)

Docker se utiliza a través de Google Cloud en una máquina virtual con Debian. En esta máquina, Docker Compose se ejecuta como servicio, permitiendo su funcionamiento en segundo plano. La selección de estas imágenes garantiza el cumplimiento de los requisitos del módulo de sistemas distribuidos, facilitando la gestión de versiones de la base de datos, la interacción entre la API y el frontend, así como la ejecución eficiente del algoritmo de IA. En conjunto, esto asegura una experiencia coherente y sin interrupciones para los usuarios finales.

En la implementación, se adoptó un enfoque estratégico basado en el modelo de comunicación cliente-servidor, a través de una arquitectura de micro servicios, en la cuál se ejecutan en contenedores las imágenes anteriormente mencionadas, logrando comunicarse unos a otros, proporcionando una estructura eficiente para la interacción de los usuarios con la plataforma, la cuál garantiza la estabilidad del sistema en situaciones adversas. Se logró establecer una comunicación efectiva entre los dispositivos de los usuarios y el servidor, donde los usuarios realizan consultas permitiendo que el servidor responda proporcionando la información de las mascotas de manera oportuna y efectiva. Este diseño facilita una interacción fluida y efectiva entre los usuarios y el servidor central, asegurando una experiencia óptima para aquellos que buscan adoptar una mascota, brindándoles acceso instantáneo a la información que están buscando.

En cuanto a la selección de protocolos de comunicación, se ha realizado una cuidadosa justificación. Se optó por el protocolo HTTP para las solicitudes web, aprovechando su amplia compatibilidad y eficiencia en entornos online. Además, se implementaron protocolos personalizados para la comunicación entre clientes y el servidor, decisiones respaldadas por consideraciones de eficiencia y seguridad. Estas elecciones están orientadas a garantizar una comunicación robusta y segura entre los usuarios y el servidor, promoviendo así la confianza y la integridad en la interacción en línea. En resumen, la adopción del modelo cliente-servidor, la efectiva comunicación entre dispositivos y las elecciones de protocolos se alinean estratégicamente para ofrecer una plataforma "RUFO" eficiente, segura y orientada al usuario.

IV. Resultados Obtenidos del Proyecto

La implementación exitosa del sistema cumplió con los objetivos de ser una plataforma eficiente de adopción en línea, que permita tener una gestión interna administrativa de mascotas adoptables, la cuál incluye el catálogo funcional de mascotas, diseñado con una interfaz de usuario intuitiva, que mejora la experiencia de los usuarios al facilitar la búsqueda y visualización de perros disponibles para adopción. Por otra parte, la gestión interna administrativa permite un seguimiento eficiente de los perros rescatados y simplifica el proceso de encontrar hogares amorosos. Además, el proyecto logra aumentar la conciencia pública sobre la adopción responsable y la importancia de gestionar la población canina.

Gracias a su diseño simple, entendible y de fácil manejo para los usuarios, así como la integración de recomendaciones y la implementación de un sistema tolerante a fallos, no solo han fortalecido la robustez del sistema, sino que también ha permitido enriquecer la interacción entre posibles adoptantes, mejorando la calidad del proceso de adopción.

V. Conclusiones y trabajo a futuro

El proyecto, materializado en el sitio web "RUFO", ha alcanzado con éxito su objetivo central al ofrecer una plataforma dedicada a la gestión eficiente de la adopción de perros, abordando de manera efectiva la preocupante sobrepoblación de caninos sin hogar y la generación de conciencia sobre la adopción responsable.

El impacto social de "RUFO" se manifiesta en la optimización de la gestión interna administrativa de mascotas adoptables y la generación de conciencia sobre la adopción responsable. Estos logros subrayan no sólo el cumplimiento de los objetivos iniciales, sino también el establecimiento de un sólido precedente para futuros proyectos que aspiren a abordar problemáticas sociales cruciales con un enfoque centrado en la seguridad y el bienestar.

Mirando hacia el futuro, con el compromiso con la causa de la adopción responsable, se contempla la implementación de métodos de pago destinados a facilitar donaciones, fortaleciendo así la capacidad de recaudación de fondos para respaldar el cuidado y bienestar de los perros. Además, se planea enriquecer la oferta de "RUFO" con un apartado de eventos, brindando la oportunidad de destacar actividades especiales, como asimismo, la creación de un blog dedicado a fascinantes temas sobre mascotas que busque fomentar la conciencia y la educación en el cuidado de los perros.

Reconocimientos

Se reconoce y expresa sincera gratitud al Mtro. [omitido] , a [omitido] , junto con la Mtra. [omitido] , por su destacado liderazgo y dirección, su visión estratégica, habilidades de gestión y dedicación, los cuales han sido fundamentales para el progreso positivo y la consecución de los objetivos establecidos.

REFERENCIAS

[1] J. Liberty y J. Galloway, Git for Programmers: Master Git for effective implementation of version control for your programming projects. Packt Publ., 2021

[2] Sinnaps. (s.f.). Scrum Planning. Sinnaps Project Management Blog. https://www.sinnaps.com/en/project-management-blog/scrum-planning

[3] Climer, Siobhan. (May, 2019). “Client-Server Model Improves Tech Management”. Recuperado de https://gomindsight.com/insights/blog/client-server-model-improves-tech-management/

[4] Vasiliauskas, Vidas. (20 February, 2024). “Scrum Advantages and Disadvantages in 2024”. Recuperado de https://teamhood.com/agile/scrum-advantages-disadvantages/

[5] Metral Kenneth. (April, 2024). “The Power of Color Psychology in Web Design”. Recuperado de https://cosmicostudios.medium.com/the-power-of-color-psychology-in-web-design-b798e956797a#:~:text=At%20the%20heart%20of%20color's,feelings%20of%20excitement%20or%20urgency.

[6] Cambronero, C. G., & Moreno, I. G. (2006). “Algoritmos de aprendizaje: knn & kmeans”. Inteligencia en Redes de Comunicación, Universidad Carlos III de Madrid

[7] Ahmed, M., Seraj, R., & Islam, S. M. S. (2020). “The k-means algorithm: A comprehensive survey and performance evaluation.” Electronics, 9(8), 1295

[8] Alake, Richmond. (January 2023). “Cosine Similarity". Builtin. Recuperado de https://builtin.com/machine-learning/cosine-similarity

[9] Aitechtrend. (February, 2023). “How Cosine Similarity Can Improve Your Machine Learning Models”. Recuperado de https://aitechtrend.com/how-cosine-similarity-can-improve-your-machine-learning-models/

[10] Docker. “Docker: Accelerated Container Application Development”. Recuperado de https://www.docker.com/

[11] Docker. What is a Container?. Recuperado de https://www.docker.com/resources/what-container/

[12] Sematext. “What Is Docker and Why Shold You Use It”. Recuperado https://sematext.com/glossary/docker/#:~:text=Configuration%20and%20consistent%20delivery%20of,%2C%20library%2C%20and%20compatibility%20issues.